Publication-grade brain visualization in Python

This guide covers every major Python tool for generating journal-quality brain surface and volume figures, with exact APIs, working code, and journal specifications. **surfplot** is the current gold standard for static publication figures, **nilearn** dominates volume-slice and quick surface plotting, and **PyVista** / **pycortex** fill specialized niches. The guide is organized tool-by-tool with complete function signatures, then by figure type and journal requirements. All code targets Python 3.11+ and reflects the 2024–2026 API state.

---

surfplot: the gold standard for publication brain figures

surfplot (v0.2.0, by Daniel Gale) GitHub wraps BrainSpace's VTK rendering into matplotlib figures. GitHub +2 It produces the cleanest multi-view surface plots with minimal code. Install with `pip install surfplot` (requires brainspace ≥0.1.1, matplotlib ≥3.3, nibabel ≥3.0, vtk ≥8.1). Readthedocs Readthedocs Recommended extras: `neuromaps` for surface fetching, `nilearn` for data projection. Readthedocs

Plot() constructor — complete signature

```python
from surfplot import Plot

p = Plot(
    surf_lh=None,           # str, PathLike, or BSPolyData — left hemisphere
    surf_rh=None,           # str, PathLike, or BSPolyData — right hemispher
    layout='grid',          # 'grid' | 'column' | 'row'
    views=None,             # str or list: 'lateral','medial','dorsal','vent
    mirror_views=False,     # flip RH view order in row/column layouts
    flip=False,             # flip LH/RH display order
    size=(500, 400),        # VTK rendering space (pixels), NOT matplotlib f
    zoom=1.5,               # camera zoom level
    background=(1, 1, 1),   # RGB background (white default)
    label_text=None,        # dict with keys 'left','right','top','bottom'
    brightness=0.5          # plain surface shade (0=black, 1=white)
)
```

`layout` **behavior:** `'grid'` produces a views × hemisphere matrix (default 2×2 for lateral+medial). `'row'` arranges all panels horizontally. `'column'` stacks vertically. Readthedocs `views` defaults to `['lateral', 'medial']` when None. Readthedocs The `size` parameter controls VTK pixel dimensions (not the final figure size — that is set via `build(figsize=...)`). Readthedocs

add_layer() — stacking data overlays

```python
p.add_layer(
    data,                    # ndarray, dict{'left':arr,'right':arr}, file p
    cmap='viridis',          # any matplotlib colormap name or object
    alpha=1,                 # layer opacity (0-1), added in v0.2.0
    color_range=None,        # tuple (min, max) for colormap; None = auto
    as_outline=False,        # draw only contiguous-region boundaries (for p
    zero_transparent=True,   # 0-valued vertices become NaN (transparent) —
    cbar=True,               # show colorbar for this layer
    cbar_label=None          # string label for colorbar
)
```

Critical behaviors: When `data` is a single numpy array, vertices are assumed **left-then-right** order (e.g., 64,984 for fsLR 32k bilateral). Readthedocs Readthedocs The dict format `{'left': lh_arr, 'right': rh_arr}` is safer and allows single-hemisphere data. Readthedocs `zero_transparent=True` (default) is essential — it automatically makes medial wall and sub-threshold vertices invisible. Readthedocs Set to `False` only when 0 is a meaningful data value. Layers stack in add order; later layers render on top.

build() — generating the matplotlib figure

```python
fig = p.build(
    figsize=None,            # matplotlib figure size (inches)
    colorbar=True,           # draw colorbars for applicable layers
    cbar_kws=None,           # dict of colorbar keyword arguments
    scale=(2, 2)             # VTK render scale factor (higher = crisper)
)
# Returns: matplotlib.figure.Figure
```

`cbar_kws` **dictionary** controls all colorbar appearance: Readthedocs Readthedocs

```python
cbar_kws = {
    'location': 'bottom',       # 'left' | 'right' | 'top' | 'bottom'
    'label_direction': None,    # angle in degrees (0=horizontal, 45=angled,
    'n_ticks': 3,               # number of colorbar ticks
    'decimals': 2,              # decimal places for tick labels
    'fontsize': 10,             # font size for labels and ticks
    'draw_border': True,        # draw tick marks and border
    'outer_labels_only': False, # labels only on outermost colorbar
    'aspect': 20,               # ratio of long to short dimension
    'pad': 0.08,                # space between colorbars
    'shrink': 0.3,              # fraction to multiply colorbar size
    'fraction': 0.05            # fraction of axes for colorbar
}
```

Utility functions

```python
from surfplot.utils import threshold, add_fslr_medial_wall
from surfplot.datasets import load_example_data

# Threshold: sets values below thresh to 0 (→ transparent with zero_transpar
thresholded = threshold(data_array, thresh=3.0)

# Add medial wall back to HCP data (59,412 → 64,984 vertices)
full_data = add_fslr_medial_wall(data_59k)
lh, rh = add_fslr_medial_wall(data_59k, split=True)

# Example data (Neurosynth 'default mode' association map on fsLR 32k)
lh_data, rh_data = load_example_data()          # separate hemispheres
data = load_example_data(join=True)               # concatenated
fronto = load_example_data('frontoparietal', join=True)
```

Loading surfaces for surfplot

```python
# fsLR surfaces via neuromaps (preferred for HCP-style data)
from neuromaps.datasets import fetch_fslr
surfaces = fetch_fslr(density='32k')  # also '4k', '8k', '164k'
lh, rh = surfaces['inflated']         # also 'midthickness', 'veryinflated',
sulc_lh, sulc_rh = surfaces['sulc']   # sulcal depth for shading

# fsaverage via neuromaps
from neuromaps.datasets import fetch_fsaverage
surfaces = fetch_fsaverage(density='164k')  # also '3k', '10k', '41k'
lh, rh = surfaces['inflated']              # also 'pial', 'white', 'sphere'

# Conte69 via brainspace (same as fsLR 32k midthickness)
from brainspace.datasets import load_conte69
surf_lh, surf_rh = load_conte69()

# Any arbitrary surface file
from brainspace.mesh.mesh_io import read_surface
surf = read_surface('path/to/surface.gii')
```

Key limitation: `fetch_fslr()` does NOT provide pial surfaces — only midthickness, inflated, veryinflated, and sphere. Netneurolab For pial, use `fetch_fsaverage()` or supply your own files. No flat surfaces are available through either fetcher; flat surfaces must come from FreeSurfer or HCP processing.

Complete working examples

Example 1 — Stat map with sulcal shading:

```python
from neuromaps.datasets import fetch_fslr
from surfplot import Plot
from surfplot.datasets import load_example_data

surfaces = fetch_fslr()
lh, rh = surfaces['inflated']
p = Plot(surf_lh=lh, surf_rh=rh)

# Sulcal depth layer (no colorbar)
p.add_layer({'left': surfaces['sulc'][0], 'right': surfaces['sulc'][1]},
            cmap='binary_r', cbar=False)

# Statistical data layer
lh_data, rh_data = load_example_data()
p.add_layer({'left': lh_data, 'right': rh_data}, cmap='YlOrRd_r',
            cbar_label='Association (z)')

fig = p.build()
fig.savefig('stat_map.png', dpi=400, bbox_inches='tight')
```

Example 2 — Parcellation with outlines:

```python
from brainspace.datasets import load_parcellation
p = Plot(lh, rh)
lh_parc, rh_parc = load_parcellation('schaefer')
p.add_layer({'left': lh_parc, 'right': rh_parc}, cmap='viridis', cbar=False)
p.add_layer({'left': lh_parc, 'right': rh_parc}, cmap='gray',
            as_outline=True, cbar=False)
fig = p.build()
```

Example 3 — Thresholded volumetric data projected to surface:

```python
from neuromaps.transforms import mni152_to_fslr
from surfplot.utils import threshold
from nilearn.plotting.cm import _cmap_d as nilearn_cmaps

gii_lh, gii_rh = mni152_to_fslr(nifti_img)
data_lh = threshold(gii_lh.agg_data(), 3)
data_rh = threshold(gii_rh.agg_data(), 3)

p = Plot(lh, rh)
p.add_layer({'left': surfaces['sulc'][0], 'right': surfaces['sulc'][1]},
            cmap='binary_r', cbar=False)
p.add_layer({'left': data_lh, 'right': data_rh},
            cmap=nilearn_cmaps['cold_hot'], color_range=(-6, 6))
fig = p.build()
```

Example 4 — Embedding into existing matplotlib figure:

```python
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
p = Plot(surf_lh=lh, surf_rh=rh)
p.add_layer(data, cmap='viridis')
ax = p.build_axis(ax)
fig.savefig('embedded.pdf', bbox_inches='tight')
```

Known gotchas and compatibility issues

VTK 9.4.0 breaks brainspace — the `load_conte69()` function crashes. GitHub Pin VTK to an earlier version if affected. GitHub `size` **vs** `figsize` **confusion** is common: `size` controls VTK rendering pixels, `figsize` in `build()` controls the matplotlib figure inches. Custom `ListedColormap` objects with parcellations can produce erroneous boundaries due to a BrainSpace rendering issue; perceptually linear cmaps mask this. GitHub The package has **not been updated since December 2022** (v0.2.0), but remains functional. Zenodo Always verify vertex counts match between data and surface mesh. The `build()` method must be called after all `add_layer()` calls.

---

nilearn: the versatile workhorse for surface and volume plots

nilearn (v0.13.x, actively maintained) provides the broadest set of plotting functions for both surface and volume data. Its volume plotting (`plot_stat_map`, `plot_glass_brain`) is unmatched; surface plotting is functional but produces slightly lower-quality renders than surfplot due to matplotlib's face-based triangle coloring vs VTK's vertex-interpolated rendering.

Surface plotting functions

`plot_surf_stat_map()` — the primary surface statistical map function:

```python
from nilearn.plotting import plot_surf_stat_map

fig = plot_surf_stat_map(
    surf_mesh=None,          # .gii, FreeSurfer file, [coords,faces], InMemo
    stat_map=None,           # 1D array, SurfaceImage, or file path
    bg_map=None,             # sulcal depth for shading (SurfaceImage or arr
    hemi='left',             # 'left', 'right', or 'both' (0.11.0+)
    view=None,               # 'lateral','medial','dorsal','ventral','anteri
                             # or (elevation, azimuth) tuple
    engine='matplotlib',     # 'matplotlib' or 'plotly'
    threshold=None,          # float, 'auto' (80th percentile), or None
    cmap='RdBu_r',
    colorbar=True,
    vmin=None, vmax=None,
    symmetric_cbar='auto',
    bg_on_data=False,        # multiply stat map by bg map (shows sulcal dep
    title=None,
    output_file=None,        # .png, .pdf, .svg — saves and returns figure
    axes=None, figure=None
)
```

`plot_surf_roi()` — for parcellation/integer label overlays:

```python
from nilearn.plotting import plot_surf_roi
plot_surf_roi(surf_mesh, roi_map, hemi='left', view='lateral',
              bg_map=sulcal, bg_on_data=True, cmap='gist_ncar')
```

`plot_img_on_surf()` — the most convenient multi-view surface function:

```python
from nilearn.plotting import plot_img_on_surf
fig, axes = plot_img_on_surf(
    stat_map,                    # 3D NIfTI (auto-projected to surface)
    surf_mesh='fsaverage5',
    hemispheres=['left','right'],
    views=['lateral','medial'],  # rows in output montage
    inflate=False,
    threshold=1.96, cmap='cold_hot',
    colorbar=True, symmetric_cbar=True,
    output_file='surface_montage.png'
)
```

`view_surf()` — interactive HTML (exploration, not journals):

```python
from nilearn.plotting import view_surf
view = view_surf(surf_mesh, surf_map=data, threshold='90%', cmap='cold_hot')
view.save_as_html('interactive.html')
```

Loading fsaverage data in nilearn

```python
# Modern API (0.11.0+) — returns PolyMesh objects
from nilearn.datasets import load_fsaverage, load_fsaverage_data
meshes = load_fsaverage('fsaverage5')       # dict: meshes['pial'], meshes['
sulcal = load_fsaverage_data(data_type='sulcal')    # SurfaceImage
curv = load_fsaverage_data(data_type='curvature')   # SurfaceImage

# SurfaceImage from volume (0.11.0+)
from nilearn.surface import SurfaceImage
surface_image = SurfaceImage.from_volume(mesh=meshes['pial'], volume_img=sta

# Legacy API (still works)
from nilearn.datasets import fetch_surf_fsaverage
fsaverage = fetch_surf_fsaverage('fsaverage5')
# Keys: pial_left, pial_right, infl_left, infl_right, white_left, white_righ
#        sulc_left, sulc_right, curv_left, curv_right, flat_left, flat_right

# Loading arbitrary surface data files
from nilearn.surface import load_surf_data, load_surf_mesh
data = load_surf_data('file.mgh')       # .mgh, .mgz, .gii, .annot, .label
coords, faces = load_surf_mesh('lh.pial')
```

fsaverage resolution table:

Name	Vertices/hemisphere	Total	Use case
fsaverage3	642	1,284	Debugging
fsaverage4	2,562	5,124	Very low-res
fsaverage5	**10,242**	20,484	Default for quick visualization
fsaverage6	40,962	81,924	Medium resolution
fsaverage (7)	**163,842**	327,684	Publication quality

Volume plotting functions

`plot_stat_map()` — the most-used volume visualization function:

```python
from nilearn.plotting import plot_stat_map

display = plot_stat_map(
    stat_map_img,                # 3D NIfTI
    bg_img='MNI152',             # background anatomical (None for black)
    display_mode='ortho',        # 'ortho','tiled','mosaic','x','y','z','xz'
    cut_coords=None,             # int (auto), list [x,y,z], or tuple
    threshold=None,              # float, 'auto', or None
    cmap='cold_hot',
    colorbar=True,
    vmin=None, vmax=None,
    symmetric_cbar='auto',
    radiological=False,          # False = neurological (left-is-left), True
    dim='auto',                  # background dimming (-2 to 2)
    draw_cross=True,
    annotate=True,               # L/R and mm annotations
    output_file=None,
    # New in 0.12.0:
    transparency=None,           # NIfTI for alpha blending
    transparency_range=None      # [low, high] for gradual transparency
)
# Save with DPI control:
display.savefig('output.png', dpi=300)
display.close()
```

Lightbox/montage — multiple slices in one direction:

```python
plot_stat_map(img, display_mode='z', cut_coords=10, threshold=3.0)  # 10 aut
plot_stat_map(img, display_mode='z', cut_coords=[-20,-10,0,10,20,30,40,50,60
```

`plot_glass_brain()` — maximum intensity projection:

```python
from nilearn.plotting import plot_glass_brain
display = plot_glass_brain(stat_img, threshold='auto', display_mode='lyrz',
                           plot_abs=False, colorbar=True)
display.add_contours(stat_img, filled=True, levels=[3.0], colors='r')
```

`plot_connectome()` — network nodes and edges on glass brain:

```python
from nilearn.plotting import plot_connectome
plot_connectome(
    adjacency_matrix,        # (n,n) array
    node_coords,             # (n,3) MNI coordinates
    node_color='auto', node_size=50,
    edge_cmap='RdBu_r',
    edge_threshold='95%',    # show top 5% of connections
    display_mode='lyrz',
    alpha=0.7, colorbar=True
)
```

Display object overlay methods (works with plot_anat, plot_stat_map, etc.): Nilearn

```python
display = plotting.plot_anat(anat_img)
display.add_overlay(stat_img, threshold=3, cmap='hot')
display.add_contours(roi_img, levels=[0.5], colors='r', linewidths=2)
display.add_markers(coords, marker_color='cyan', marker_size=50)
display.title("Custom title", size=16)
display.savefig('composite.png', dpi=300)
display.close()
```

How nilearn compares to surfplot

nilearn's surface rendering uses matplotlib's `plot_trisurf` (face-based coloring), which produces visible triangle artifacts at close zoom. Neurostars surfplot uses VTK vertex-interpolated rendering, yielding smoother results. GitHub PyPI surfplot also has superior multi-panel layout support (`Plot(lh, rh, views=['lateral','medial'])` vs nilearn's single-view-per-axes approach) and native multi-layer stacking. However, nilearn's volume plotting is unmatched (surfplot has no volume capabilities), its dependency footprint is lighter, and its data loading/projection pipeline is more complete. **Use surfplot for surface figures, nilearn for volume slices and quick exploration.**

PyVista: full 3D control when surfplot is not enough

PyVista (actively maintained, v0.47+, `pip install pyvista`) provides complete low-level 3D rendering via VTK. Use it when you need custom camera angles, animations, non-standard surfaces, or interactive exploration.

Loading FreeSurfer surfaces

```python
import nibabel as nib
import numpy as np
import pyvista as pv

vertices, faces = nib.freesurfer.read_geometry('lh.inflated')
faces_pv = np.column_stack([np.full(len(faces), 3), faces]).ravel()
mesh = pv.PolyData(vertices, faces_pv)

# Attach scalar overlay
thickness = nib.freesurfer.read_morph_data('lh.thickness')
mesh['thickness'] = thickness
```

Rendering with publication theme

```python
pv.set_plot_theme('document')  # white background, clean fonts
plotter = pv.Plotter(off_screen=True, window_size=[1600, 1200])
plotter.add_mesh(mesh, scalars='thickness', cmap='viridis', clim=(1.0, 4.0),
                 smooth_shading=True, show_scalar_bar=True,
                 scalar_bar_args={'title': 'Thickness (mm)', 'n_labels': 5,
                                  'title_font_size': 16, 'label_font_size':
plotter.camera_position = [(-300, 0, 0), (0, 0, 0), (0, 0, 1)]  # lateral le
plotter.screenshot('lateral.png', transparent_background=True)
plotter.close()
```

Standard camera positions

```python
views = {
    'lateral_left':  [(-300, 0, 0), (0, 0, 0), (0, 0, 1)],
    'lateral_right': [(300, 0, 0),  (0, 0, 0), (0, 0, 1)],
    'medial_left':   [(300, 0, 0),  (0, 0, 0), (0, 0, 1)],
    'medial_right':  [(-300, 0, 0), (0, 0, 0), (0, 0, 1)],
    'dorsal':        [(0, 0, 300),  (0, 0, 0), (0, 1, 0)],
    'ventral':       [(0, 0, -300), (0, 0, 0), (0, 1, 0)],
    'anterior':      [(0, 300, 0),  (0, 0, 0), (0, 0, 1)],
    'posterior':     [(0, -300, 0), (0, 0, 0), (0, 0, 1)],
}
```

Each tuple is `(camera_location, focal_point, view_up_vector)`. PyVista Adjust the **300** distance to fit your brain's bounding box.

Other surface visualization tools

brainplotlib — zero-dependency 2D brain plots

brainplotlib renders pre-computed 2D brain surface images as NumPy arrays using only numpy + matplotlib. GitHub **No VTK or OpenGL required** — ideal for HPC/remote servers. Fixed lateral+medial 4-view layout on fsaverage surfaces only. Install: `pip install brainplotlib`. Last release v0.6.3 (2023), low maintenance.

```python
from brainplotlib import brain_plot
img, scale = brain_plot(vertex_data, vmax=10, vmin=-10, cmap='seismic',
                        return_scale=True, surface='inflated')
# img is RGBA NumPy array (H×W×4)
import matplotlib.pyplot as plt
plt.imshow(img); plt.axis('off'); plt.colorbar(scale)
```

enigmatoolbox — parcellation-to-surface mapping

The ENIGMA Toolbox (Nature Methods, 2021) excels at mapping ROI-level statistics to surface vertices, with built-in support for DK, Glasser, and Schaefer parcellations plus **subcortical 3D visualization**. Install: `pip install enigmatoolbox`.

```python
from enigmatoolbox.plotting import plot_cortical, plot_subcortical
from enigmatoolbox.utils.parcellation import parcel_to_surface

# Map 68 DK ROI values to surface vertices
vertex_data = parcel_to_surface(roi_values, 'aparc_fsa5')
plot_cortical(array_name=vertex_data, surface_name='fsa5',
              cmap='RdBu_r', color_range=(-0.5, 0.5), color_bar=True)

# Subcortical (14 structures: thalamus, hippocampus, amygdala, etc.)
plot_subcortical(array_name=subcortical_values, cmap='RdBu_r', ventricles=Tr
```

Supported parcellation strings: `aparc_fsa5`, `glasser_fsa5`, `schaefer_100_fsa5` through `schaefer_400_fsa5`, and their `_conte69` equivalents. Moderate maintenance — core functionality stable but not heavily developed since 2023.

ggseg-python — 2D polygon brain schematics

Python port of R's ggseg. Produces 2D outline-style brain plots, not 3D surface renders. GitHub **Inactive since 2021** (v0.1). Snyk Only supports DK, aseg, and JHU atlases. Libraries.io The R original is far more feature-rich. For Python parcellation visualization, prefer enigmatoolbox or surfplot.

```python
import ggseg
ggseg.plot_dk(data_dict, cmap='Spectral', figsize=(15,15),
              background='k', edgecolor='w', ylabel='Thickness (mm)')
```

pycortex — the flatmap specialist

pycortex is the gold standard for cortical flatmap visualization. Requires FreeSurfer `recon-all` output and a subject database. **Linux/macOS only** (no Windows). GitHub

```python
import cortex
volume = cortex.Volume('data', subject='S1', xfmname='retinotopy',
                       vmin=-2, vmax=2, cmap='RdBu_r')
cortex.quickflat.make_figure(volume, with_curvature=True, with_rois=True)
```

Flatmap generation requires FreeSurfer's `mris_flatten` (~2 hours/hemisphere) or Blender UV unwrap (5–15 minutes). Gallantlab

netplotbrain — network-on-brain visualization

Specialized for rendering nodes and edges on brain templates. Published in Network Neuroscience (2023). Integrates with TemplateFlow. MIT Press

```python
import netplotbrain
netplotbrain.plot(nodes=node_df, edges=edge_df,
                  node_size='degree', node_color='community',
                  template='MNI152NLin2009cAsym', template_style='glass',
                  view=['LSR'])
```

neuromaps — data transforms, not visualization

neuromaps (Nature Methods, 2022) handles brain map fetching, coordinate transforms, and spatial statistics. Libraries.io It does **not** do visualization itself but is essential for preparing data for surfplot/nilearn.

```python
from neuromaps.transforms import mni152_to_fslr, fsaverage_to_fslr, fslr_to_
from neuromaps.datasets import fetch_fslr, fetch_fsaverage, fetch_annotatior

fslr_lh, fslr_rh = mni152_to_fslr(nifti_img, '32k')    # volume → fsLR surfa
fslr = fsaverage_to_fslr(fsavg_tuple, '32k')             # fsaverage → fsLR
# For parcellations, use method='nearest':
fslr = fsaverage_to_fslr(label_data, '32k', method='nearest')
```

Cerebro Viewer — interactive CIFTI/GIFTI viewer

Early-stage (v0.0.9+) interactive 3D brain viewer built on VTK. Natively supports CIFTI, GIFTI, and NIfTI. GitHub Documentation is sparse but improving. Install: `pip install Cerebro-Viewer`.

FreeSurfer file handling in Python

Loading every FreeSurfer file type with nibabel

```python
import nibabel as nib

# Surfaces: .pial, .inflated, .white, .sphere, .orig
vertices, faces = nib.freesurfer.read_geometry('lh.pial')
vertices, faces, vol_info = nib.freesurfer.read_geometry('lh.pial', read_met

# Morphometry: .thickness, .curv, .sulc, .area, .volume (binary per-vertex s
data = nib.freesurfer.read_morph_data('lh.thickness')  # returns 1D ndarray

# Annotations: .annot (parcellation labels + color table)
labels, ctab, names = nib.freesurfer.read_annot('lh.aparc.annot')
# labels: vertex-to-label array; ctab: n_labels × 5 (RGBT+id); names: list c

# Labels: .label (ROI vertex indices)
vertices = nib.freesurfer.read_label('lh.V1.label')
vertices, scalars = nib.freesurfer.read_label('lh.V1.label', read_scalars=Tr

# MGH/MGZ overlays (volume or surface data)
img = nib.load('lh.thickness.fwhm10.fsaverage.mgh')
data = img.get_fdata().squeeze()  # (n_vertices,) for surface data

# Writing
nib.freesurfer.write_geometry('output.surf', coords, faces)
nib.freesurfer.write_morph_data('output.curv', values)
nib.freesurfer.write_annot('output.annot', labels, ctab, names)
```

Medial wall handling

The medial wall comprises **non-cortical vertices** on the medial surface (corpus callosum region). In fsLR 32k, each hemisphere has 32,492 total vertices; HCP data typically contains **29,696 cortical vertices per hemisphere** (59,412 total), with ~2,796 medial wall vertices excluded. Readthedocs

```python
# surfplot: zero_transparent=True (default) handles it automatically
p.add_layer(data, zero_transparent=True)  # 0-valued → transparent

# surfplot utility for adding medial wall back to HCP data
from surfplot.utils import add_fslr_medial_wall
full_data = add_fslr_medial_wall(hcp_data_59k)        # 59,412 → 64,984
lh, rh = add_fslr_medial_wall(hcp_data_59k, split=True)

# Manual NaN masking
import numpy as np
data = np.array(vertex_data, dtype=float)
data[medial_wall_mask] = np.nan
```

Surface templates at a glance

Template	Vertices/ hemi	Space	Load with
fsaverage (7)	163,842	FreeSurfer	`fetch_fsaverage('164k')` or `load_fsaverage('fsaverage')`
fsaverage6	40,962	FreeSurfer	`load_fsaverage('fsaverage6')`
fsaverage5	**10,242**	FreeSurfer	`load_fsaverage('fsaverage5')` — quick visualization default
fsLR 32k	**32,492**	HCP/ Conte69	`fetch_fslr('32k')` — HCP standard
fsLR 164k	163,842	HCP	`fetch_fslr('164k')`

Volume-to-surface and surface-to-volume conversion

Volume to surface (well-supported)

```python
# nilearn (recommended)
from nilearn.surface import vol_to_surf
from nilearn.datasets import fetch_surf_fsaverage
fsaverage = fetch_surf_fsaverage('fsaverage5')
surf_data = vol_to_surf(stat_img, surf_mesh=fsaverage['pial_left'],
                        inner_mesh=fsaverage['white_left'])  # depth samplin

# Modern nilearn API (0.11+)
from nilearn.surface import SurfaceImage
from nilearn.datasets import load_fsaverage
meshes = load_fsaverage('fsaverage5')
surface_image = SurfaceImage.from_volume(mesh=meshes['pial'], volume_img=sta

# neuromaps (for fsLR space)
from neuromaps.transforms import mni152_to_fslr
fslr_lh, fslr_rh = mni152_to_fslr(nifti_img, '32k')
data_lh = fslr_lh.agg_data()
```

Surface to volume (requires workarounds)

nilearn does **not** have a `surf_to_vol` function — this is a known gap. Three approaches exist:

FreeSurfer mri_surf2vol (via command line):

```bash
mri_surf2vol --o output.nii.gz --subject bert \
  --so $SUBJECTS_DIR/bert/surf/lh.white $SUBJECTS_DIR/bert/surf/lh.thickness
  --so $SUBJECTS_DIR/bert/surf/rh.white $SUBJECTS_DIR/bert/surf/rh.thickness
```

Python ribbon-mask approach (manual but flexible):

```python
import nibabel as nib
import numpy as np
from scipy.spatial import cKDTree

ribbon = nib.load('mri/ribbon.mgz')
ribbon_data = ribbon.get_fdata()
vertices, faces = nib.freesurfer.read_geometry('surf/lh.white')
thickness = nib.freesurfer.read_morph_data('surf/lh.thickness')

vol = np.zeros(ribbon_data.shape)
cortex_mask = (ribbon_data == 3)  # LH cortex voxels (3=LH cortex, 42=RH cor
voxel_coords = np.argwhere(cortex_mask)
ras_coords = nib.affines.apply_affine(ribbon.affine, voxel_coords)
tree = cKDTree(vertices)
_, nearest_vtx = tree.query(ras_coords)
vol[cortex_mask] = thickness[nearest_vtx]

out_img = nib.Nifti1Image(vol, ribbon.affine, ribbon.header)
nib.save(out_img, 'thickness_in_volume.nii.gz')
```

Nipype wrapper for mri_surf2vol:

```python
import nipype.interfaces.freesurfer as fs
surf2vol = fs.Surface2VolTransform()
surf2vol.inputs.source_file = 'lh.thickness'
surf2vol.inputs.hemi = 'lh'
surf2vol.inputs.subject_id = 'bert'
surf2vol.run()
```

Neurological vs radiological convention

Neurological convention (left-is-left) is standard in neuroimaging and preferred by most journals. **Radiological convention** (left-is-right) is traditional in clinical radiology. NIfTI files store orientation in the affine matrix.

```python
import nibabel as nib
img = nib.load('brain.nii.gz')
print(nib.aff2axcodes(img.affine))  # ('R','A','S') = neurological; ('L','A'

# Convert to canonical RAS (neurological)
img = nib.as_closest_canonical(img)

# nilearn: radiological=False (default) = neurological convention
from nilearn.plotting import plot_stat_map
plot_stat_map(img, radiological=False)  # left-is-left (default)
```

Standard neuroimaging figure types and how to create each

Vertex-wise statistical maps (t-maps, z-maps)

The most common brain figure. Use **surfplot** for best quality: load inflated surface, add sulcal shading layer (cmap='binary_r', cbar=False), add thresholded stat map layer with diverging colormap. Standard 4-view grid layout (lateral L, medial L, lateral R, medial R). Threshold conventions: **p < 0.05 FDR** for rigorous analysis, **p < 0.001 uncorrected** for liberal display. Color convention: hot (red/yellow) for positive, cool (blue/cyan) for negative; use `RdBu_r` or nilearn's `cold_hot`.

Parcellation-based heatmaps

Map ROI-level statistics to surface vertices, then render. enigmatoolbox's `parcel_to_surface()` is the easiest approach. Alternatively:

```python
import numpy as np
labels, ctab, names = nib.freesurfer.read_annot('lh.Schaefer2018_400Parcels.
vertex_data = np.zeros(len(labels), dtype=float)
for idx, value in enumerate(parcel_values):
    vertex_data[labels == idx] = value
```

Network-level visualizations with Yeo canonical colors

The Yeo 7-network assignment uses these canonical RGB values from FreeSurfer's color lookup table:

Network	Label	RGB	Hex
Visual	7Networks_1	(120, 18, 134)	#781286
Somatomotor	7Networks_2	(70, 130, 180)	#4682B4
Dorsal Attention	7Networks_3	(0, 118, 14)	#00760E
Ventral Attention	7Networks_4	(196, 58, 250)	#C43AFA
Limbic	7Networks_5	(220, 248, 164)	#DCF8A4
Frontoparietal	7Networks_6	(230, 148, 34)	#E69422
Default Mode	7Networks_7	(205, 62, 78)	#CD3E4E

```python
from matplotlib.colors import ListedColormap
import numpy as np
yeo7_colors = np.array([[120,18,134],[70,130,180],[0,118,14],
                        [196,58,250],[220,248,164],[230,148,34],[205,62,78]
yeo7_cmap = ListedColormap(yeo7_colors, 'yeo7', N=7)
```

Glass brain with connectome

```python
from nilearn.plotting import plot_connectome
plot_connectome(corr_matrix, node_coords, node_size=degree_array*10,
                node_color=community_colors, edge_threshold='95%',
                edge_cmap='RdBu_r', display_mode='lyrz', alpha=0.7)
```

Convention: node size ∝ effect size or degree; edge color = correlation strength; edge width = connection weight.

Multi-panel composite figures

```python
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec

fig = plt.figure(figsize=(183/25.4, 120/25.4), dpi=600)  # Nature double-col
gs = gridspec.GridSpec(2, 3, width_ratios=[1, 1, 0.8], hspace=0.3, wspace=0.

# Brain panel (use surfplot's build_axis)
ax_brain = fig.add_subplot(gs[0, :2])
p = Plot(lh, rh)
p.add_layer(data, cmap='RdBu_r')
p.build_axis(ax_brain)

# Bar plot panel
ax_bar = fig.add_subplot(gs[0, 2])
ax_bar.bar(x, heights, color=colors)

# Panel labels
fig.text(0.02, 0.95, 'a', fontsize=8, fontweight='bold')
fig.text(0.68, 0.95, 'b', fontsize=8, fontweight='bold')
fig.savefig('composite.pdf', dpi=600, bbox_inches='tight')
```

Manual orthogonal volume slices with matplotlib

```python
import nibabel as nib
import numpy as np
import matplotlib.pyplot as plt

img = nib.as_closest_canonical(nib.load('brain.nii.gz'))
data = img.get_fdata()
mni_coord = np.array([0, -20, 40])
vox = np.round(nib.affines.apply_affine(np.linalg.inv(img.affine), mni_coord

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(np.rot90(data[vox[0], :, :]), cmap='gray'); axes[0].set_title
axes[1].imshow(np.rot90(data[:, vox[1], :]), cmap='gray'); axes[1].set_title
axes[2].imshow(np.rot90(data[:, :, vox[2]]), cmap='gray'); axes[2].set_title
for ax in axes: ax.axis('off')
plt.savefig('slices.png', dpi=300, bbox_inches='tight')
```

Journal figure specifications

Quick reference table

Journal	Single col	Double col	Min DPI	Font min	Preferred formats	Color
Nature	89 mm	183 mm	300 (600 combo)	5 pt	AI/EPS/PDF/TIFF	RGB
NeuroImage	~85 mm	~180 mm	300/500/1000	6–7 pt	TIFF/EPS/PDF	RGB
Brain	90 mm	185 mm	300 (600 line)	—	TIFF	RGB
PNAS	8.7 cm	17.8 cm (1.5 col: 11.4 cm)	300/600/1000	6 pt	TIFF/EPS/PDF	RGB
eLife	Flexible	Flexible	300 rec.	—	Any (digital-first)	RGB

Nature-specific requirements

Panel labels must be **8 pt bold lowercase** (a, b, c — not italic). Font must be **Helvetica or Arial** throughout, minimum **5 pt**. Vector formats (PDF, EPS, AI) preferred for graphs. RGB color mode recommended. Maximum page depth **247 mm**. Nature explicitly requires colorblind-accessible palettes (cites Wong, Nature Methods 2011) and prohibits red-green contrast combinations.

PNAS-specific requirements

Panel labels are **capital letters** (A, B, C, D) in upper-left corner. Approved fonts: Arial, Helvetica, Times, Symbol. Minimum **6 pt** after reduction, maximum **12 pt**. All figures must be pre-assembled composite files, not embedded in manuscript.

Saving for journals

```python
# Vector format (best for most journals)
fig.savefig('figure.pdf', bbox_inches='tight')

# High-res raster
fig.savefig('figure.tiff', dpi=600, bbox_inches='tight')
fig.savefig('figure.png', dpi=400, bbox_inches='tight', transparent=True)

# Nature double-column figure setup
fig = plt.figure(figsize=(183/25.4, 120/25.4))  # mm to inches
plt.rcParams.update({'font.family': 'Arial', 'font.size': 7})

# surfplot-specific export
fig = p.build(figsize=(7.2, 5), scale=(3, 3))  # higher scale = crisper VTK
fig.savefig('figure.pdf', bbox_inches='tight')
```

Colorblind-safe palettes for neuroimaging

Nature, NeuroImage, and most top journals now require or strongly recommend colorblind-accessible figures. **Never use jet or rainbow** — they create false perceptual bands, fail under color vision deficiency, and convert to non-monotonic grayscale.

Recommended sequential colormaps: `viridis` (general-purpose default), `cividis` (optimized for all CVD types), `inferno`, `magma`, `plasma`. **Recommended diverging colormaps:** `RdBu_r` (nilearn/surfplot default), `coolwarm`, `BrBG`. For checking colorblind safety, use Coblis (coblis.com), the Python `colorspacious` package for CVD simulation, or Crameri et al.'s scientific colour maps (fabiocrameri.ch/colourmaps — perceptually uniform and CVD-certified). The Crameri colormaps are installable via `pip install cmcrameri` and integrate directly with matplotlib.

Tool selection decision tree

- **Surface stat map for a paper** → surfplot (4-view grid, sulcal shading, clean colorbars)
- **Quick volume slice figure** → nilearn `plot_stat_map` (unmatched convenience)
- **Volume lightbox/montage** → nilearn `plot_stat_map(display_mode='z', cut_coords=10)`
- **Parcellation ROI values on surface** → enigmatoolbox `parcel_to_surface` + surfplot
- **Glass brain / connectome** → nilearn `plot_glass_brain` or `plot_connectome`
- **Cortical flatmap** → pycortex
- **Network nodes + edges on template** → netplotbrain
- **Custom 3D rendering / animation** → PyVista
- **Interactive HTML exploration** → nilearn `view_surf()` or Cerebro Viewer
- **No GPU / HPC server** → brainplotlib (numpy-only rendering)
- **Subcortical 3D structures** → enigmatoolbox `plot_subcortical`
- **Cross-space data transforms** → neuromaps (then visualize with surfplot/nilearn)
- **2D brain schematics** → ggseg (if R ggseg not available; Python version is unmaintained)

Import statement summary

python

```python
# === surfplot ===
from surfplot import Plot
from surfplot.utils import threshold, add_fslr_medial_wall
from surfplot.datasets import load_example_data

# === nilearn plotting ===
from nilearn.plotting import (plot_surf_stat_map, plot_surf_roi, plot_img_on
                              plot_stat_map, plot_glass_brain, plot_anat,
                              plot_connectome, plot_roi, view_surf, show)
from nilearn.datasets import (load_fsaverage, load_fsaverage_data,
                              fetch_surf_fsaverage, fetch_atlas_surf_destri
                              fetch_atlas_schaefer_2018)
from nilearn.surface import (vol_to_surf, load_surf_data, load_surf_mesh, Su

# === neuromaps ===
from neuromaps.datasets import fetch_fslr, fetch_fsaverage, fetch_annotation
from neuromaps.transforms import (mni152_to_fslr, mni152_to_fsaverage,
                                  fsaverage_to_fslr, fslr_to_fsaverage)

# === brainspace ===
from brainspace.datasets import load_conte69, load_parcellation
from brainspace.mesh.mesh_io import read_surface

# === nibabel FreeSurfer ===
import nibabel as nib
# nib.freesurfer.read_geometry, read_morph_data, read_annot, read_label

# === PyVista ===
import pyvista as pv

# === enigmatoolbox ===
from enigmatoolbox.plotting import plot_cortical, plot_subcortical
from enigmatoolbox.utils.parcellation import parcel_to_surface

# === pycortex ===
import cortex

# === colormaps ===
from matplotlib.colors import ListedColormap
from nilearn.plotting.cm import _cmap_d as nilearn_cmaps  # e.g., 'cold_hot'
```

Conclusion

The Python neuroimaging visualization ecosystem in 2026 is mature but fragmented. **surfplot remains the single best tool for static publication figures** — its `Plot → add_layer → build` pipeline produces journal-ready 4-view surface plots in under 10 lines of code. **nilearn is indispensable for volume visualization** and retains the broadest feature set. For specialized tasks — flatmaps (pycortex), connectomes (netplotbrain), parcellation mapping (enigmatoolbox), custom 3D (PyVista) — purpose-built tools outperform general ones. The critical workflow pattern is: transform data with neuromaps, load surfaces from fetch_fslr/fetch_fsaverage, render with surfplot, export at ⩾300 DPI in PDF/TIFF with Arial font at ⩾5 pt. Always use colorblind-safe colormaps (viridis for sequential, RdBu_r for diverging), and always add sulcal depth shading as the base layer for anatomical context.